MODUL PRAKTIKUM 7
EXCEPTION HANDLING



Disusun Oleh:
Ananda Firdaus (251511003)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2025

Repo: https://github.com/FirdausSinau/PBO

BAGIAN A - CountLetters

A.1 Code awal

Program membaca satu kata, mengubahnya menjadi huruf besar, lalu menghitung frekuensi setiap huruf. Index array diperoleh dari `word.charAt(i) - 'A'` tanpa pengecekan karakter.

Code A.1. CountLetters versi awal.

```
import java.util.Scanner;

public class CountLetters {
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);

        System.out.print("Enter a single word (letters only, please): ");
        String word = scan.nextLine();
        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++)
            counts[word.charAt(i) - 'A']++;

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " + counts[i]);
    }
}
```

Output A.1. Input `JAVA`.

```
Enter a single word (letters only, please): JAVA

A: 2
J: 1
V: 1
```

Huruf A muncul dua kali, J dan V satu kali. Huruf A sampai Z dipetakan ke index 0 sampai 25 melalui `'A'` = 65.

A.2 Bukti ArrayIndexOutOfBoundsException

Output A.2. Input `JAVA PBO!` sebelum ada catch.

```
Enter a single word (letters only, please): JAVA PBO!

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index -33
out of bounds for length 26
    at CountLetters.main(CountLetters.java:13)
```

Spasi berkode 32 menghasilkan index `32 - 65 = -33`. Exception terjadi di baris 13. Karena tidak ada handler, `main` berhenti sebelum loop.

A.3 Code try-catch

Code A.2. Body loop dibungkus try-catch dengan handler kosong.

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
    }
}
```

Output A.3. Input `JAVA PBO!` setelah catch kosong.

```
Enter a single word (letters only, please):
A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Program tidak berhenti. Spasi dan `!` dilewati, huruf setelahnya tetap dihitung.

A.4 Output setelah handler diperbaiki

Code A.3. CountLetters final.

```
import java.util.Scanner;

public class CountLetters {
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);

        System.out.print("Enter a single word (letters only, please): ");
        String word = scan.nextLine();
        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++) {
            try {
                counts[word.charAt(i) - 'A']++;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Not a letter: " + word.charAt(i));
            }
        }

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " + counts[i]);
    }
}
```

Output A.4. Input `JAVA PBO!` setelah handler diperbaiki.

```
Enter a single word (letters only, please): Not a letter:
Not a letter: !

A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Baris pertama berisi dua spasi setelah titik dua, satu dari teks dan satu dari spasi. Loop tetap berjalan karena `try-catch` berada di dalam `for`. Exception buang satu iterasi.

BAGIAN B - ParseInts

B.1 try-catch di luar while

Output B.1. try-catch di luar while.

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 0

Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 1
```

Pada input pertama, token `We` gagal sehingga belum ada angka yang dijumlahkan. Pada input kedua, angka `1` sempat masuk, lalu `fish` gagal dan angka `2` tidak dibaca. Loop berhenti karena exception keluar dari `try` yang membungkus seluruh while.

B.2 try-catch di dalam while

Output B.2. try-catch di dalam while.

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 3

Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 3

Enter a line of text
10 20 30 40
The sum of the integers on this line is 100
```

Setiap token diproses sendiri. Token yang gagal membuang satu iterasi, sehingga `2` dan `1` tetap dijumlahkan.

B.3 Perbedaan alur

Penempatan	Cakupan try	Saat token invalid	Token berikutnya
Di luar while	seluruh loop	keluar dari try dan while	Ga diproses
Di dalam while	satu iterasi	berpindah ke catch	diproses

Perbedaan hasil berasal dari luas blok `try`, bukan si tipe exception. `try` yang mencakup seluruh loop membuat satu kegagalan menghentikan loop. `try` yang mencakup satu iterasi membuat kegagalan terbatas pada token itu.

BAGIAN C - Factorials

C.1 Hasil baseline

Code C.1. MathUtils versi awal.

```
public class MathUtils {
    public static int factorial(int n) {
        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

Output C.1. Baseline input 5, 0, -1, dan 17.

```
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n) y
Enter an integer: 0
Factorial(0) = 1
Another factorial? (y/n) y
Enter an integer: -1
Factorial(-1) = 1
Another factorial? (y/n) y
Enter an integer: 17
Factorial(17) = -288522240
Another factorial? (y/n) n
```

`factorial(-1)` mengembalikan 1 karena loop `i > 0` tidak pernah berjalan untuk `n = -1`. `factorial(17)` menghasilkan negatif karena 17! melebihi kapasitas `int`. Nilai 1 untuk input negatif tidak bisa dibedakan dari hasil yang sah.

C.2 Code throw IllegalArgumentException

Code C.2. factorial menolak nilai negatif.

```
public static int factorial(int n) throws IllegalArgumentException {
    if (n < 0) {
        throw new IllegalArgumentException(
            "factorial tidak didefinisikan untuk bilangan negatif: " + n);
    }

    int fac = 1;
    for (int i = n; i > 0; i--)
        fac *= i;
    return fac;
}
```

C.3 Exception sebelum catch

Output C.2. Input -1 tanpa catch.

```
Enter an integer: -1
Exception in thread "main" java.lang.IllegalArgumentException: factorial tidak
didefinisikan untuk bilangan negatif: -1
    at MathUtils.factorial(MathUtils.java:4)
    at Factorials.main(Factorials.java:12)
```

Exception dibuat di `MathUtils.factorial` baris 4, terus naik ke `Factorials.main` baris 12. Tanpa catch, program berenti.

C.4 Code handler

Code C.3. Handler di dalam loop.

```
while (keepGoing.equals("y") || keepGoing.equals("Y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();

    try {
        System.out.println("Factorial(" + val + ") = "
                + MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Tidak dapat menghitung: " + e.getMessage());
    }

    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```

`scan.nextInt()` berada di luar `try` agar handlernya cuman menangani exception dari perhitungan.

C.5 Output input valid, negatif, dan lebih dari 16

Code C.4. MathUtils final.

```
public class MathUtils {
    public static int factorial(int n) throws IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException(
                "nilai tidak boleh negatif: " + n);
        }
        if (n > 16) {
            throw new IllegalArgumentException(
                "nilai terlalu besar untuk factorial berbasis int: " + n);
        }

        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

Output C.3. Pengujian input 5, 0, -1, 16, dan 17.

```
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n) y
Enter an integer: 0
Factorial(0) = 1
Another factorial? (y/n) y
Enter an integer: -1
Tidak dapat menghitung: nilai tidak boleh negatif: -1
Another factorial? (y/n) y
Enter an integer: 16
Factorial(16) = 2004189184
Another factorial? (y/n) y
Enter an integer: 17
Tidak dapat menghitung: nilai terlalu besar untuk factorial berbasis int: 17
Another factorial? (y/n) n
```

Input	Hasil	Keterangan
5	120	benar
0	1	benar
-1	message negatif, loop lanjut	ditolak
16	2004189184	diterima sesuai batas modul
17	message terlalu besar, loop lanjut	ditolak

BAGIAN D - Tabel Skenario Uji

ID	Skenario	Expected Result	Hasil	Status	Bukti
T7-01	CountLetters dengan satu kata	Frekuensi huruf tercetak tanpa exception	A:2 J:1 V:1	PASS	A.1
T7-02	CountLetters dengan spasi atau tanda baca sebelum catch	ArrayIndexOutOfBoundsException terlihat	Index -33 out of bounds for length 26	PASS	A.2
T7-03	CountLetters setelah catch	Karakter non-huruf dilewati dan proses lanjut	Not a letter: spasi dan !, semua huruf tercetak	PASS	A.4
T7-04	ParseInts try-catch di luar while	Proses berhenti pada token invalid pertama	Sum 0 dan sum 1	PASS	B.1
T7-05	ParseInts try-catch di dalam while	Semua token diproses, integer tetap dijumlahkan	Sum 3, 3, dan 100	PASS	B.2
T7-06	factorial(-1) setelah throw, sebelum catch	IllegalArgumentException menghentikan alur	Exception dan program berhenti	PASS	C.3
T7-07	factorial(-1) setelah catch di loop	Message tampil dan program dapat lanjut	Message negatif, lanjut ke 5 dan menghasilkan 120	PASS	C.5
T7-08	factorial(17)	Exception dengan message batas lebih dari 16 ditangani	Message terlalu besar, loop lanjut	PASS	C.5

BAGIAN E - Refleksi

1. Pada CountLetters, mengapa program awal berhenti saat menemukan karakter non-huruf? Apa yang berubah setelah ada catch?

Program awal gak meriksa apakah karakter benar-benar huruf. Spasi menghasilkan index -33, dan tanpa handler, `main` berhenti. Setelah `catch` ditambahkan di dalam loop, karakter non-huruf dilewati dan loop tetap lanjut.

2. Apakah exception selalu berarti aplikasi harus berhenti? Gunakan satu bukti dari praktikum.

Tidak. Pada ParseInts, token `We` melempar `NumberFormatException`, tetapi program tetap memproses `2` dan `1` sehingga sum menjadi 3. Pada Factorials, input -1 ditolak dan input 5 tetap dihitung.

3. Mengapa try-catch di luar while pada ParseInts menghasilkan behavior berbeda dengan try-catch di dalam while?

`try` di luar while mencakup seluruh loop, sehingga exception keluar dari loop dan token berikutnya tidak diproses. `try` di dalam while hanya mencakup satu iterasi, sehingga setelah `catch` loop lanjut ke token berikutnya.

4. Dalam kasus Factorials, mengapa melempar exception lebih aman daripada mengembalikan nilai yang terlihat valid tetapi sebenarnya salah?

`return 1` untuk -1 terlihat okeh karena `factorial(0)` juga bernilai 1. Pemanggil tidak tahu terjadi kegagalan dan bisa memakai angka salah. `throw` menghentikan method tanpa mengembalikan nilai, sehingga kegagalan tidak bisa diabaikan.

5. Apa perbedaan peran throw dan catch berdasarkan code yang Anda jalankan?

`throw` berada di method dan berfungsi melaporkan input tidak valid. `catch` berada di pemanggil dan berfungsi memutuskan apakah program lanjut atau berhenti.

6. Apa manfaat memberikan message berbeda pada IllegalArgumentException untuk nilai negatif dan nilai >16?

Kedua kondisi melempar tipe exception yang sama, sehingga masuk ke `catch` yang sama. Message membedakan penyebabnya: nilai negatif atau nilai terlalu besar. Pengguna jadi tahu input mana yang harus diperbaiki.

7. Ketiga exception utama pada modul ini termasuk unchecked exception. Apa yang ingin Anda tanyakan ketika nanti membandingkannya dengan checked exception?

Yang ingin saya tanyakan: apa itu checked exception?

8. Lengkapi dengan bahasa Anda sendiri: "Exception handling bukan sekadar mencegah program crash, tetapi ..."

Exception handling bukan sekadar mencegah program crash, tetapi memisahkan masalah dari keputusan untuk melanjutkan, jadi kegagalan gak berubah menjadi nilai palsu.